《UEFI 内核导读》之 ACPI 编程入门

做知识的搬运工

固件 C 字营·出品

Cstyle·编著

样 本

目录

序 3
第 1 章：ACPI 规范简介 4
第 2 章：名词术语（Term） 5
第 3 章：Hello World 6
第 4 章：方法（Method） 7
第 5 章：对象（Object） 8
第 6 章：对象类型及命名 9
第 7 章：特殊对象与类型 11
第 8 章：类型转换及规则 12
第 9 章：常用 ASL 算子（operator）简介 14
第 10 章：ACPI 表 15
第 11 章：ACPI 硬件 16
第 12 章：ACPI 操作系统 17
第 13 章：Low Power S0 Idle 简介 18
第 14 章：ACPI/ASL 调试简介 19

序

随着国家十四五新战略规划的推出，众多国内企业都参与到国产芯片替代浪潮中来，可以预测未来越多的国产芯片会被设计、生产和使用在我们日常所使用的电子产品中，国产芯片拥有巨大的市场前景。

目前国产芯片采用的体系架构主要有 X86、ARM、MIPS、RISC V、PowerPC、Alpha 等。我们知道电子产品正常工作必须要有操作系统和各种应用软件，没有操作系统和应用软件的芯片就是一堆废铁，而大多数人并不知道的是没有系统固件来加载操作系统的电脑亦是一堆废铁， UEFI 就是由 UEFI 行业协会提出和维护一种行业标准的系统固件，它支持目前市面上的大多数芯片体系结构和操作系统，随着标准的不断演进相信越来越多的体系结构的芯片和操作系统会被支持。

笔者从事 BIOS 开发已有十余年的时间，见证了 Legacy BIOS 辉煌与隐退，也有幸了参与了新世纪初系统固件从 Legacy BIOS 往 UEFI BIOS 的迁移的全过程。科技行业风起云涌新技术新架构日新月异，每每回望不禁感慨我辈可谓是“眼见着他起高楼，眼见着他宴宾客”的那一波 BIOS 人。曾经系统固件江湖还是 Legacy BIOS 的天下，BIOS 人使用汇编语言编码、通过中断来与操作系统沟通。自 UEFI 框架被广泛使以来开我们的发环境从纯汇编变成了 99%的 C 语言加 1%的汇编语言的模式，开发效率大大的加强了。

虽然 UEFI 框架大大加快了开发效率，但是由于系统固件开发属于比较偏门和专业的领域，学习和入门门槛比较高，现有的 BIOS 工程师又分布在大大小小的各个公司内部缺乏有效沟通和交流，同时 BIOS 源码又属于敏感和机密数据受到各种 NDA 限制，市面上对 UEFI 框架介绍的资料少之又少，因此笔者从 2000 左右开始就陆续以 Cstyle_0x007 为 ID 在 https://blog.csdn.net/CStyle_0x007 发布一系列博文，现已有数十篇原创文章。刚开始的想法是把博文当作工作笔记方便自己随时查阅，后来慢慢发展成了与业内外感兴趣的朋友的沟通交流的平台。

随手写的博文难免有错误与纰漏为了避免误导大众，准备把博文重新整理在纠正谬误同时也会补充一些新的内容，尽量做到所写的每句话都是无误的，也欢迎有兴趣的朋友踊跃提出意见和建议。组建了微信公众号，目的在于方便有兴趣的朋友一起交流，名字初步定为“固件 C 字营”，其中“固件”泛指一切固化的软件，这里主要指 UEFI BIOS 系统固件，“C”泛指“China“，我们可以把这里当作大家沟通交流的营地，我们会不定时发布一些行业资讯、工作、学习心得，感兴趣扫描下面二维码就可以加入，也可以发邮件到 CstyleFirmWareCamp@outlook.com 投稿分享你的想法。

本文取名《UEFI 内核的导读》这里的 UEFI 专指“UEFI BIOS”,全文专注于对 UEFI 内核的梳理与分享，同时兼顾对 X86 系统固件生态中常用的工程技术的介绍，主要包含以下内容：UEFI 启动流程以及各个阶段主要完成的任务及参考的实现方式导读 UEFI 及 PI 规范中的常见 Protocol 的实现与使用技巧 UEFI 固件生态中常见外设、总线、行业标准的协议内容及使用方法

雄关漫道真如铁，而今迈步从头越，系统固件雄起之路道阻且长，相信我们的 BIOS 人一定可以为国产芯片的起飞助力、为系统固件团队的壮大贡献自己的一份微薄之力，为每一个不畏艰难、不惧寂寞坚守在工作岗位的 BIOS 人加油，好样的。

固件 C 字营・武汉

11/21/2020

第 1 章：ACPI 规范简介

ACPI（Advanced Configuration and Power Interface）是由 intel 及微软等企业在 1990 年左右提出的 PC 电源管理接口规范。其目标是把操作系统和系统固件（UEFI/BIOS）隔离开来，使操作系统和硬件（固件）厂商可以在共同标准下相互解耦独立发展。ACPI 最新的版本是 6.4，下文内容都是基于 6.4 的版本进行说明。

ACPI 规范包含了 ACPI 硬件、ACPI 软件、ACPI 表、设备管理、电源管理、热管理等内容。其中 ASL（ACPI Source Language）语言是默认用来编写 ACPI 代码的“声明式编程语言”，AML（ACPI Machine Language）是 ASL 源码被编译器编译后被 AML 解释器解释执行的字节码。AML 是所有 ACPI 兼容的操作系统必须支持的，但是 ASL 是非必须的，只要是能最终被转换成 AML 的其他的任何语言都可以用来编写 ACPI 代码。

ASL 用来定义 ACPI 对象(object)和 ACPI 控制方法（Method），ASL 支持预定义的内置算子和宏供程序（一般是 BIOS 工程师编写）或 OS 调用，如：数学运算、逻辑运算等。

ACPI 规范定义了大量的与硬件和 OS 相关的内容，整个文档洋洋洒洒有数千页之多，阅读起来甚是难以理解。本文试图以 BIOS 工程师的视角来对其进行简单的梳理，参照一般的编程语言如“C 语言“类似的讲解逻辑从“Hello World”开始，介绍如何使用 ACPI/ASL 编写自己的程序，如何编译、运行、调试代码。以及 ACPI 表、ACPI 硬件、ACPI 操作系统、系统睡眠唤醒流程等。

第 2 章：名词术语（Term）

ACPI：Advanced Configuration and Power Interface
ASL：ACPI Source Language,经过编译之后生成 AML 字节码
AML: ACPI Machine Language
方法：Control Method，控制方法，类似 C 语言的函数，以下称之为“方法”。
对象：Object 对象用来对一类 ACPI 实体进行抽象，本文不区分“变量”和“对象”。
参数：一般指传递给 Method 的，不同类型的对象
OSPM：Operating System-directed configuration and Power Management
ArgX：代表 Arg0-Arg6，共 7 个 Method 输入变量
LocalX：代表 Local0-Local7，共 8 个局部变量
宏：ASL Macro，编译器内置的功能，类似 C 语言的宏，在编译后会扩展成算子或对象集
算子：ASL operator，用来对 ACPI 对象或数据进行操作和运算的动作和方法
操作数：ASL operand，被算子操作的对象或数据
数据类型：Data type，用来区分不同的对象，每一类对象都有其唯一的数据（对象）类型
编译器：一般指 ACPI/ASL 编译器如：iASL
解释器：用来解释执行 AML 字节码的程序，是 OSPM 的一部分
调试终端：可供系统打印字符的设备，如串口、特定的内存空间、WinDebug 等

第 3 章：Hello World

第一个 ACPI/ASL 程序“Hello World”，该程序会在调试终端输出“Hello World!”字符串。
第一行，表示该程序需要引用外部的方法（Method）。
第二行，空行，编译器会自动忽略。
第三行，表示定义一个方法“HOWD()“，该方法用“大括号”表示其作用域。
第四/六行，空行，编译器会直接忽略，每一条代码以行结尾。
第五行，调用 ADBG()方法从调试终端输出字符串。

```
1  External(\ADBG, MethodObj)
2
3  Method(HOWD,1)
4  {
5      ADBG("Hello World!")
6  }
```

以上便是最简单的 ACPI/ASL 语言“Hello World”程序，同时它也是最复杂的程序，为什么这么说呢？原因是：

1. ASL 代码一般必须要固化到 BIOS ROM 内部，所以你必须要把 ASL 代码根 BIOS 源码一起　　编译进 BIOS，并且烧到主板上才可以执行。
2. 这里只写了一个 Method，具体如何调用 Method 并没有详细说明，因为 ACPI 当中并没有一个类似 C 语言的统一的 Main()函数入。当然要想调用这个 Method 我们有很多的办法比如：放在特定的 ACPI 设备内部，或自定义一个 ACPI 设备然后在 OS 下通过 OS 提供的接口直接调用 ACPI Method，或把 Method 放在系统睡眠/唤醒调用流程中等等不一而足，后面会详细说明。
3. ACPI/ASL 语言写的代码如何调试？调试方法又很多，比如：人肉调试，直接写出没错误的代码、用终端打印、插桩法、用 WinDebug 单步调试等等

调试方法见 第 14 章：ACPI/ASL 调试简介

第 4 章：方法（Method）

Control Method 是 ACPI/ASL 预定义的一种对象类型（ObjectType）为 8 的对象。它可以调用或者被其他的 Method 调用，在调用过程中可以被阻塞但不能被抢占，当一个 Method 被阻塞时 OSPM 可以转而执行其他的 Method。以下称之为“方法“或“Method”。

一、输入参数：

Method 支持输入最多 7 个输入参数（Arg0- Arg6）或无输入参数，ArgX 分三种类型。其中控制方法类（control method）和表达式（ASL expression）类的 ArgX 在 Method 对其使用完后，自动对其进行删除操作。三种 ArgX 类型分别为：

1. 命名的对象（named object）：把不同类型的对象当做参数传递给 Method
2. 控制方法（control method）：执行 ArgX 绑定的方法，把返回值当参数传入到调用 Method
3. 表达式（ASL expression）：先计算表达式，再把结果传入 Method。

二、输入参数传递

Method 调用时参数采用常量引用的方式传递（call-by-reference-constant），意味着在调用过程中不会复制参数，而是直接引用参数，且参数不能被修改，这样既减少了在 OS 启动过程中对内存的频繁访问，也可以防止对对象的意外修改。在 Method 内部修改 ArgX 不会修改到调用前的原对象。

如果使用 RefOf 算子来引用对象，并通过 ArgX 传递到 Method 内部，这个时候对引用对象到修改就会覆盖原始对象，也就达到了对原始数据进行修改目的。

当 Buffer 对象(ObjectType=3)或 Package 对象(ObjectType=4)被当做 ArgX 传递到 Method，使用间接的 Index 或 Field 去访问 Buffer 或 Package 内部成员时会覆盖其内部数据。

三、局部变量。

Method 支持最多 8 个局部变量对象（Local0-Local7）,其初始值为 NULL。可以使用预定义的 Name (ObjectName, Object)方法来在全局 ACPI 名字空间（Global ACPI namespace）创建名为 ObjectName 的新对象。下例创建了 3 个对象，其作用域分别是：

```
1   Scope (\XYZ)
2   {
3       Name (BAR, 5)              // Creates \\XYZ.BAR
4       Method (FOO, 1)
5       {
6           CREG = BAR             // same effect as CREG = \XYZ.BAR
7           Name (BAR, 7)          // Creates \\XYZ.FOO.BAR
8           DREG = BAR             // same effect as DREG = \XYZ.FOO.BAR
9           Name (\XYZ.FOOB, 3)    // Creates \\XYZ.FOOB
10      }// end method
11  }
```

\\XYZ.BAR：静态对象，该对象所在地 ASL 被 OSPM 加载之后就一直存在

\\XYZ.FOO.BAR：动态对象，第#7 行 Name (BAR, 7)执行完之后才被创建

\\XYZ.FOOB：动态对象，第#9行Name (XYZ.FOOB,3)执行完后被创建，method退出后就被毁。

四、用途

Method 可以访问系统中的各种资源或操作空间（Operation Region），也可以进行数学运算及其他运算，进而完成特定的任务。操作空间包括：内存、IO、PCI 配置空间、EC、SMBUS、CMOS、PCIBar，IPMI、GPIO、SerialBus、PCC、OEM 自定义空间。数学运算包括：四则运算（加减乘除）、逻辑运算（逻辑与或非）、位运算（移位、位与或非）等

更多精彩尽情期待……